Assignment 1: Requirements Engineering, Architecture & Proof of Concept

Collaborative Document Editor with AI Writing Assistant

February 2026

Context

Your team has been contracted by a startup to design and build a **real-time collaborative document editing platform** with an integrated AI writing assistant. Think of it as a simplified Google Docs competitor with embedded LLM-powered features.

Multiple users must be able to edit the same document simultaneously, see each other's changes in real time, and invoke an AI assistant that can rewrite, summarize, translate, or restructure selected portions of text. Beyond the core editing experience, the system must handle authentication and authorization, document lifecycle management, a clean API layer, and a user experience that makes collaboration intuitive.

The design decisions you make in this assignment will have cascading consequences throughout the semester. You are expected to think carefully about what you are building, how the codebase will be organized, how your team will collaborate, and how the pieces fit together—and to demonstrate feasibility through a proof-of-concept.

Diagram Requirements

All architectural diagrams in this assignment must be created using **draw.io** (also known as diagrams.net) or **Mermaid**. Submit diagrams as follows:

- **draw.io**: Export diagrams as PDF or PNG and embed them in your document. Also submit the original `.drawio` files alongside your report so we can inspect and comment on them.
- **Mermaid**: Include the Mermaid source code in a clearly labeled code block in your report, along with a rendered image (PNG or PDF). You can use the Mermaid Live Editor (https://mermaid.live) to render diagrams.

Hand-drawn diagrams, PowerPoint screenshots, or diagrams without source files will not be accepted. The goal is that diagrams remain **editable and versionable**—they are living artifacts that will evolve with your project.

Part 1: Requirements Engineering (30%)

1.1 Stakeholder Analysis

Identify and characterize at least **four distinct stakeholder categories** beyond "end user." For each stakeholder, describe their goals, concerns, and how they influence system require-

ments. Consider stakeholders across the full product lifecycle—not just the people who type in documents.

1.2 Functional Requirements

Produce a structured requirements specification covering the following capability areas. For each area, provide requirements at **two levels of detail**: a high-level capability statement and at least three precise, testable sub-requirements.

Capability areas to address:

- **Real-time collaboration**: simultaneous editing, presence awareness (who is online, where their cursor is), conflict handling when two users edit the same region
- **AI writing assistant**: text enhancement, summarization, translation, reformatting—including how the user invokes the AI, how suggestions are presented, and how they are accepted or rejected
- **Document management**: creation, versioning, sharing, access control, export to common formats
- **User management**: authentication, authorization, roles (e.g., owner, editor, commenter, viewer), session handling

For each functional requirement, specify: a unique identifier, a clear description, the triggering condition, the expected system behavior, and acceptance criteria.

1.3 Non-Functional Requirements

Address **each** of the following quality attributes with specific, measurable requirements. Do not write vague aspirations—write constraints that can be verified.

Latency
: Define latency targets for (a) keystroke propagation between collaborators, (b) AI assistant response initiation, (c) document load time. Justify each target with a user-experience argument.

Scalability
: How many concurrent editors per document must you support? How many concurrent documents system-wide? What is the expected growth model?

Availability
: What is your availability target? What happens to in-progress editing sessions during a partial system failure? How do you handle AI service unavailability gracefully?

Security & Privacy
: Document content may be sensitive. How do you handle data at rest, in transit, and during AI processing? What are the implications of sending user content to third-party LLM APIs? What is your data retention policy for AI interaction logs?

Usability
: What happens when a user opens a large document with many active collaborators? How do you prevent the UI from becoming overwhelming? What are your requirements for accessibility?

1.4 User Stories and Scenarios

Write **at least ten user stories** following standard format. These must span the full breadth of the system—not just editing scenarios. Include stories covering:

Collaboration scenarios
E.g., a user goes offline mid-edit and reconnects; two users edit the same paragraph simultaneously; a user reverts to a previous version while others are editing.

AI assistant workflows
E.g., a user selects a paragraph and asks for a summary; a user requests translation of a section into another language; a user asks the AI to restructure a document outline; the AI produces a suggestion the user wants to partially accept and partially modify.

Document lifecycle
E.g., a user shares a document with read-only access; a user exports the document with AI-suggested changes tracked separately; a team lead reviews the AI interaction history to understand how a section evolved.

Access control and roles (at least two)
E.g., a user with "commenter" role tries to invoke the AI assistant; an organization admin configures which AI features are available to different roles; a viewer attempts to edit and the system prevents it gracefully.

For stories involving non-obvious system behavior, describe the expected behavior in detail and justify your design choice.

1.5 Requirements Traceability

Provide a traceability matrix linking user stories to functional requirements to architecture components (from Part 2). This matrix must demonstrate that every user story is supported by identified requirements and that every requirement maps to at least one architectural component.

Part 2: System Architecture (45%)

2.1 Architectural Drivers

Before presenting your architecture, explicitly state the **architectural drivers**—the requirements and quality attributes that most strongly influence your design. Rank them and explain why. Two teams with different driver rankings should arrive at different architectures; if your ranking doesn't actually influence your design, you haven't thought hard enough.

2.2 System Design using the C4 Model

Present your system architecture using the **C4 model** (https://c4model.com). You must produce diagrams (in draw.io or Mermaid) for at least the following levels:

Level 1 — System Context Diagram
Show the system as a whole and its relationships with external actors (users, third-party services such as LLM APIs, identity providers, etc.). This answers the question: *what does the system interact with?*

Level 2 — Container Diagram
Zoom into the system and show the major containers (e.g., frontend application, backend API, database, AI service, real-time service). For each container, indicate its responsibility and the technology choice. Show how containers communicate with each other.

Level 3 — Component Diagram
For at least **one** container of your choice (we recommend the backend API or the AI inte-

gration service), zoom in further and show its internal components, their responsibilities, and their interactions.

Each diagram must be accompanied by a brief written explanation. Beyond the diagrams, your architecture documentation must address the following concerns:

Feature Decomposition

Break the system into clearly defined features/modules and explain the responsibility of each. At minimum, your decomposition should cover:

- The rich-text editor and its frontend state management
- The real-time synchronization layer (how edits propagate between users)
- The AI assistant service (how AI features are invoked, how suggestions flow back)
- Document storage and versioning
- User authentication and authorization
- The API layer connecting frontend and backend

For each module, describe: what it does, what it depends on, and what interface it exposes to other modules. The goal is a design where modules can be developed, tested, and evolved somewhat independently.

AI Integration Design

The AI assistant is not a simple API call bolted onto the side—it is a core product feature that touches many parts of the system. Address the following:

Context and scope
: What document context does the AI see when a user invokes it? The full document, a section, or just the selection? What are the trade-offs in terms of cost, relevance, and latency? How do you handle very long documents?

Suggestion UX
: How are AI suggestions presented to the user? Inline replacement, tracked-change-style proposals, a side panel, or something else? Can users partially accept a suggestion? Can they undo an accepted suggestion?

AI during collaboration
: What happens when a user requests an AI rewrite while others are editing the same region? Do you lock the region, show a pending state, or let edits continue and reconcile later? Describe the user experience for all parties involved.

Prompt design
: How do you construct prompts for different AI features (rewrite, summarize, translate, restructure)? Is the prompt logic hardcoded, template-based, or configurable? How would you update prompts as the product evolves without redeploying the whole system?

Model and cost strategy
: Do you use the same LLM for all features, or different models for different tasks? How do you manage cost—per-user quotas, organization-level budgets, or something else? What happens when a user exceeds their limit?

API Design

Design the API layer that connects the frontend to backend services. Provide:

- The API contract for at least: document CRUD, real-time session management, AI assistant invocation, and user/permission management. You can present these as endpoint lists, OpenAPI-style definitions, or interface descriptions—but they must be concrete, not vague.
- Your choice of API style with justification for each type of interaction.
- How you handle long-running AI operations from the client's perspective. Does the client poll? Subscribe to events? How does it know when the AI is done?
- Your error handling strategy: how do clients distinguish between "the AI is slow," "the AI failed," and "you've exceeded your quota"?

Authentication & Authorization

Your system will need to control who can access and modify documents. At a high level, describe:

- Why is authentication needed and what types of users do you expect?
- What roles exist in the system (e.g., owner, editor, viewer) and what can each role do? Think beyond basic read/write—consider actions like invoking the AI assistant, reverting versions, or sharing documents.
- What are the privacy considerations of sending document content to third-party LLM APIs?

You do not need to specify the exact authentication technology or protocol at this stage—focus on the **what and why** rather than the how.

Communication Model

Your system needs to keep multiple users in sync as they edit the same document. At a high level, describe:

- Does your system use real-time push-based communication (e.g., changes appear instantly for all users), polling, or some other approach? What are the trade-offs of your choice in terms of user experience and complexity?
- What happens when a user first opens a shared document? What happens if they lose connectivity and come back?

You do not need to commit to a specific protocol (WebSocket, SSE, etc.) at this stage—focus on describing the **communication model and its implications** for the user experience.

2.3 Code Structure & Repository Organization

This section is about how you organize the actual codebase. Provide a **repository structure diagram or tree** showing how you organize your code. Address:

- **Monorepo vs. multi-repo**: Do you keep frontend, backend, and shared code in one repository or separate ones? Justify your choice considering your team size, deployment strategy, and shared dependencies.
- **Directory layout**: Show the top-level directory structure and explain the purpose of each major directory. For example, where do API route definitions live? Where are the AI prompt templates? Where is the collaboration logic?
- **Shared code**: If frontend and backend share type definitions, validation logic, or constants, how do you organize that shared code to avoid duplication without creating tight coupling?
- **Configuration management**: Where do API keys, database connection strings, and LLM provider credentials live? How do you ensure secrets never end up in the repository?

- **Testing structure**: Where do tests live relative to source code? What kinds of tests do you plan (unit, integration, end-to-end)? How do you test the AI integration without making real API calls every time?

2.4 Data Model

Design the data model for documents, users, permissions, versions, and AI interactions. Provide an **entity-relationship diagram** (in draw.io or Mermaid) and address:

- How is a document represented in storage? What fields does it have beyond the content itself?
- How do you handle document versioning? Can users see a version history and revert to a previous version?
- How do you model the AI interaction history—linking suggestions to document context, tracking whether they were accepted, rejected, or partially applied?
- How do you model permissions and sharing? A document may be shared with individual users, teams, or via link with different permission levels.

2.5 Architecture Decision Records (ADRs)

Document your **four most consequential** design decisions as ADRs. These should span different areas of the system—not all about the same concern. Use the following structure:

Title Short descriptive name
Status Proposed / Accepted
Context What forces are at play? What is the problem?
Decision What did you decide?
Consequences What are the resulting trade-offs—both positive and negative?
Alternatives considered What did you reject and why?

Part 3: Project Management & Team Collaboration (15%)

3.1 Team Structure & Ownership

Define roles and responsibilities within your team. Address:

- Who owns which parts of the codebase? Define clear ownership areas (e.g., frontend, backend/API, AI integration, infrastructure).
- How do you handle features that span multiple owners (e.g., the AI assistant touches frontend, backend, and the collaboration layer)?
- What is your decision-making process when team members disagree on a technical choice?

3.2 Development Workflow

Describe how your team will collaborate on code day-to-day:

- **Branching strategy**: What branching model do you use (feature branches, etc.)? How do you name branches? What is your merge policy?
- **Code review**: What is your code review process? Who reviews what? What are your criteria for approving a pull request?
- **Issue tracking and task assignment**: How do you break work into tasks? How are tasks assigned to team members? How do you track what is in progress vs. done?
- **Communication**: What tools do you use for team communication? How do you document decisions so they don't get lost in chat messages?

3.3 Development Methodology

Choose and justify a development methodology. Describe your iteration structure, how you prioritize the backlog, and how you handle work that doesn't produce user-visible features (e.g., setting up infrastructure, designing the data model, writing tests).

3.4 Risk Assessment

Identify at least **five technical risks** specific to this project (not generic software risks). For each risk, provide:

- Description and likelihood assessment
- Impact analysis (what breaks if this risk materializes?)
- Mitigation strategy
- Contingency plan (what do you do if mitigation fails?)

Your risks should span multiple concerns. Include at least one risk related to AI integration (cost, latency, or reliability) and at least one related to team coordination or code organization.

3.5 Timeline and Milestones

Provide a realistic timeline for the remainder of the semester with concrete, verifiable milestones. Each milestone must have clear acceptance criteria—"finish backend" is not a milestone; "API serves document CRUD operations with authentication, verified by integration tests" is.

Part 4: Proof of Concept (10%)

Build a minimal proof-of-concept that demonstrates basic **front-end to back-end communication** and validates your core data contracts. This is not expected to be a feature-complete prototype—it is a technical skeleton that proves your team can connect the pieces and that your API design works in practice.

PoC Requirements

1. **A working frontend**: A basic page (or set of pages) that represents the document editing interface. It does not need to be polished or feature-rich—a simple text area or editor component is sufficient. The focus is on showing that the frontend exists, loads, and communicates with the backend.

2. **Front-end to back-end communication**: The frontend must make at least one meaningful API call to the backend and display the result. For example: creating a document, loading a document, or saving content. This should demonstrate that your API contracts (as defined in Section 2.2) actually work end-to-end.

3. **Data contract validation**: The API request and response formats should match what you specified in your architecture document. If your design says the document API returns a JSON object with specific fields, the PoC should demonstrate exactly that.

You are free to go beyond these minimums—for instance, adding a basic real-time sync demo, a mock AI call, or a login flow—but it is not required for this assignment. The goal is to prove that the architectural skeleton holds together, not to deliver a working product.

PoC Evaluation Criteria

- Does the frontend successfully communicate with the backend through the API?

- Do the data contracts in the code match what was specified in the architecture document?
- Is the repository organized according to the structure described in Section 2.3?
- Is there a working README that allows someone to clone the repo and run the PoC with minimal setup?
- Is the Git history clean—do commits have meaningful messages? Is there evidence that the team collaborated on the code?

PoC Deliverables

- Source code in a version-controlled repository (Git)
- A README explaining: how to set up and run the PoC, what it demonstrates, and what it intentionally does not implement yet
- A short (3 minute max) recorded demo showing the frontend communicating with the backend

Submission Format

All written deliverables should be submitted as a single document (PDF) with clearly labeled sections following the structure above. C4 diagrams and other architecture diagrams must be submitted both embedded in the document and as separate editable files (`.drawio` or `.mermaid`). The PoC source code should be submitted as a link to a Git repository. The demo recording should be linked or attached separately.

Grading Rubric

Important Notes

- **This assignment rewards depth over breadth.** A thorough analysis of a few critical design decisions is worth more than a superficial treatment of fifteen.
- **Justify everything.** "We chose X" is not architecture. "We chose X because of constraints A and B, accepting trade-off C, and rejecting Y because of limitation D" is architecture.
- **Think about how you will actually build this.** The code structure, repository layout, team ownership, and workflow sections are not busywork—they are how teams turn architecture into working software. A beautifully designed system that nobody knows how to organize or divide up is incomplete.
- **Think about the AI as a product feature, not a technical demo.** Calling an LLM API is trivial. Designing a system where AI assistance is useful, predictable, cost-effective, and well-integrated into the editing experience is not. Show us you've thought about the user experience of AI suggestions, the cost implications, and the privacy concerns.
- **The PoC must match the architecture.** A design document paired with a prototype that was clearly built independently signals that the architecture wasn't taken seriously. The module boundaries, API contracts, and data model in your code should reflect what you described on paper.
- **Diagrams are living documents.** We require draw.io or Mermaid specifically because your architecture will evolve. Diagrams that can be versioned, diffed, and updated alongside code are far more valuable than static images.

Component	Weight	Excellent	Adequate	Insufficient
Requirements Engineering	30%	Requirements are precise, testable, and span the full system. User stories cover collaboration, AI workflows, document lifecycle, and access control with thoughtful edge cases.	Requirements cover the basics but focus narrowly on one area while neglecting others.	Requirements are vague, untestable, or copied from generic templates.
System Architecture	45%	C4 diagrams are clear and span multiple levels (context, container, component). Architecture is well-decomposed into modules. AI integration, API design, code structure, and repo organization are all addressed with concrete detail.	Architecture is reasonable but C4 diagrams are incomplete or only cover one level. Some areas are hand-waved. Code structure is missing or disconnected from the design.	Architecture is a single generic diagram with no C4 structure. No evidence of thinking about feature decomposition, code organization, or how the team will actually build this.
Project Management	15%	Team roles are clear with defined code ownership. Development workflow (branching, code review, task tracking) is concrete. Risks are specific and technical. Timeline has verifiable milestones.	Generic but reasonable project plan. Development workflow is mentioned but vague.	Copy-paste project management with no project-specific content. No development workflow described.
Proof of Concept	10%	PoC demonstrates working front-end to back-end communication with data contracts matching the architecture document. Repo is well-organized with a clear README. Git history shows team collaboration.	PoC runs but data contracts don't match the architecture, or the repo is disorganized.	PoC does not run, or there is no front-end to back-end communication.